

PROJECT REVIEW PRESENTATION

Range-Aware Edge AI Quantization Compiler

SSA-Based Static Analysis for Safe FP32 → INT8 Optimization



Course: BCSE307L — Compiler Design

Yash Pradhan (24BCE0702) • Anjini Pandey (24BCE0714) • Ishita Srivastava (24BDS0234)

Faculty Guide: Prof. Kanagaraj R

Why Precision Matters at the Edge

Edge Device Constraints

M

Memory

Limited RAM/flash for weights and intermediate values.

C

Compute

Restricted CPU/accelerator throughput vs. server GPUs.

E

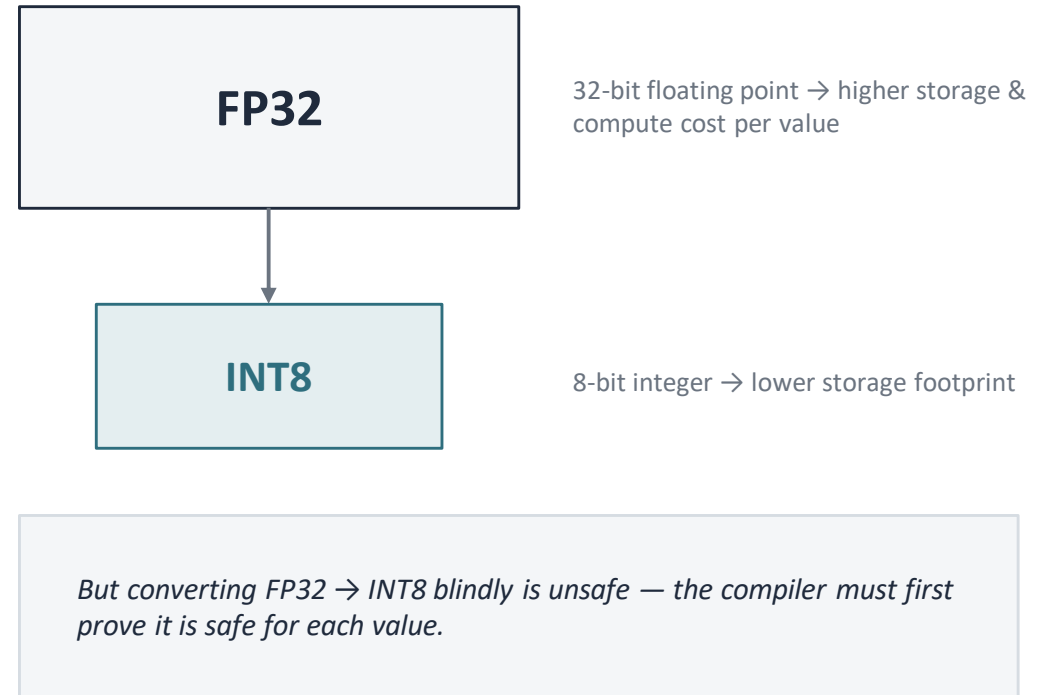
Energy

Battery-powered operation favors low-power arithmetic.

D

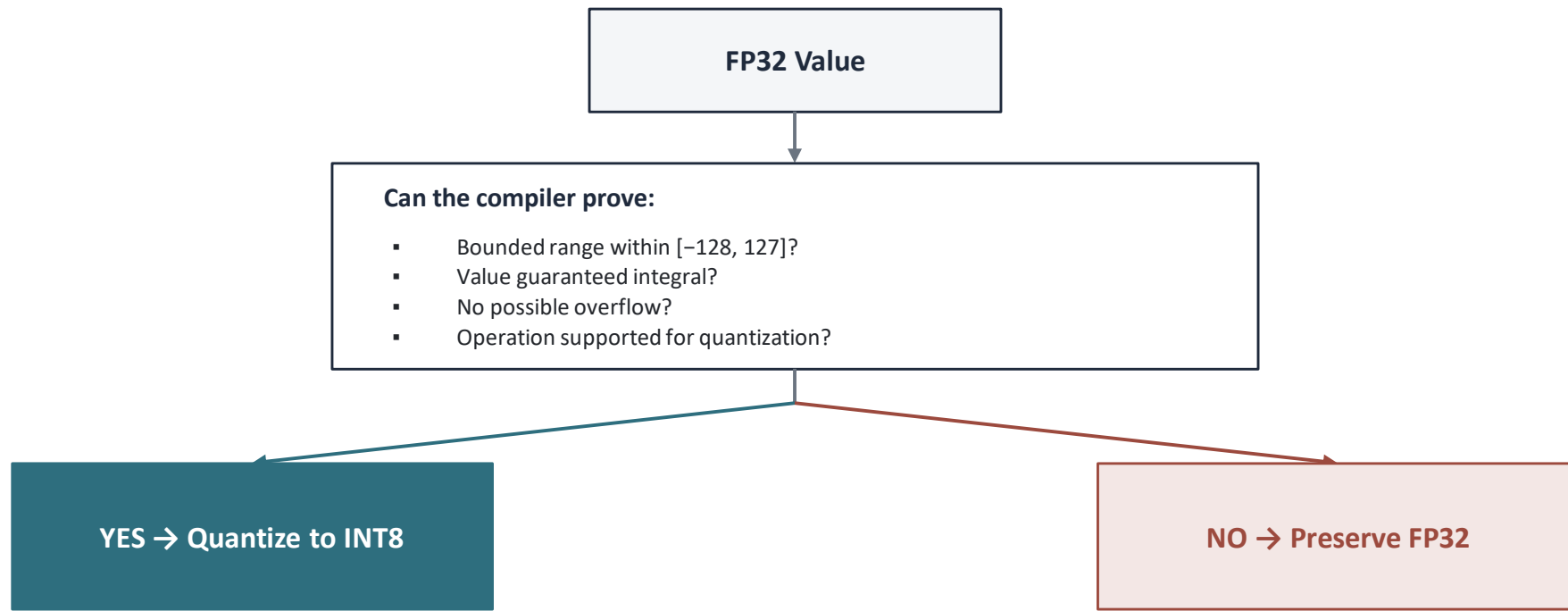
Data Movement

Off-chip memory access dominates cost in many workloads.



The Central Question

“Can a compiler statically determine which numerical computations can be safely represented using INT8, and transform only those computations for which safety can be proven?”



Why Compiler-Based Analysis?

Manual Conversion



Error-prone, difficult to scale across programs

Dynamic / Runtime Checking



Correct, but adds overhead on every execution

Static Compiler Analysis



Decision made once, before execution — our approach

The compiler inspects intermediate representation and reasons about value ranges before performing the optimization — the safety decision is explainable and made once, at compile time.

Project Objectives

1

Build a small numerical C-like compiler front end.

Lexer, parser and AST for a deliberately compact language.

2

Generate Three Address Code and construct CFG.

Lower AST to TAC and form basic blocks.

3

Convert the IR into SSA form.

Enable precise, flow-sensitive analysis.

4

Perform static range and integrality analysis.

Interval reasoning over SSA values, including loops.

5

Develop conservative INT8 safety checks.

Reject whenever safety cannot be proven.

6

Transform proven-safe computations to INT8 IR.

Quantize only what is provably safe.

7

Preserve source-level traceability throughout.

Line/column tracked from lexer to diagnostics.

Research Landscape

Edge AI / Efficient Inference

Resource-constrained deployment of ML workloads on devices.

Quantization / Reduced Precision

Representing values/weights with fewer bits (e.g. INT8).

This Project — Intersection of all four areas

Compiler-Based Optimization

Using compiler passes/IR to drive performance transformations.

Static Analysis / SSA

Flow-sensitive, provable reasoning over program values.

Key Related Work

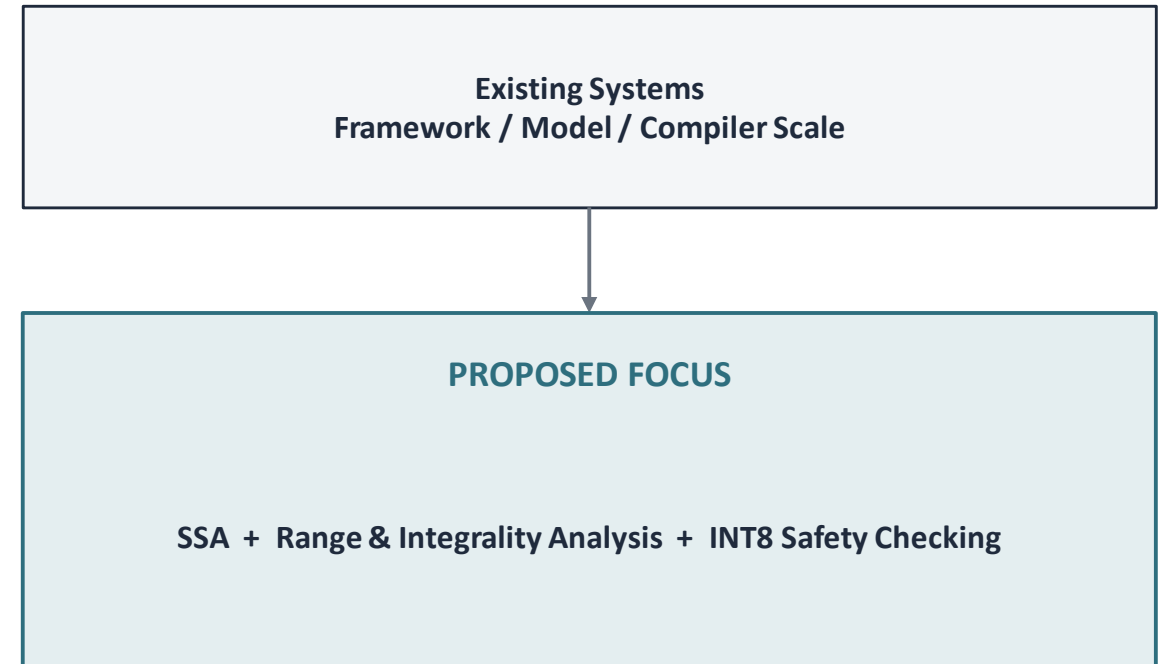
Work	Main Idea	Relevance	Difference from Our Project
Jacob et al. [1] CVPR 2018	Integer-only inference with quantization-aware training	Shows INT8 arithmetic can match FP32 accuracy	Trains a full network; we statically analyze a program IR
Krishnamoorthi [2] arXiv 2018	Practical guide to post-training & QAT quantization	Documents which value ranges are typically safe	Empirical guidelines; we derive safety via static proof
Bourdoncle [3] LNCS 1993	Widening/narrowing for chaotic fixed-point iteration	Basis for our loop-convergence / widening strategy	General technique; applied here at small, teaching scale
Blanchet et al. [9] PLDI 2003	Astrée static analyzer	Proving absence of run-time errors in embedded C	Industrial-scale range analysis; project is a compact analogue for a small language
Zhou et al. [4] Proc. IEEE 2019	Edge intelligence: survey of on-device AI	Frames the resource-constraint motivation for Edge AI	Survey/motivation only; we build a compiler pipeline
Chen et al. (TVM) [5] OSDI 2018	End-to-end compiler for portable DL execution	Shows compilers can drive precision/backend decisions	Whole-framework compiler; ours is a teaching-scale one
Cytron et al. [6] ACM TOPLAS 1991	Efficient construction of SSA form	Provides the SSA construction our pipeline relies on	General SSA algorithm; we add range/integrality analysis
Cousot & Cousot [7] POPL 1977	Abstract interpretation and fixed-point analysis	Theoretical basis for interval analysis over loops	General framework; we specialize it to INT8 safety
Lattner & Adve (LLVM) [8] CGO 2004	SSA-based IR for lifelong optimization	Shows SSA IR scales to a production compiler	Full compiler infrastructure; ours is single-purpose

[1] Jacob et al., CVPR 2018 • [2] Krishnamoorthi, arXiv:1806.08342, 2018 • [3] Bourdoncle, LNCS 735, 1993 • [4] Zhou et al., Proc. IEEE, 2019 • [5] Chen et al., OSDI 2018 • [6] Cytron et al., ACM TOPLAS, 1991 • [7] Cousot & Cousot, POPL 1977 • [8] Lattner & Adve, CGO 2004 • [9] Blanchet et al., PLDI 2003

Where This Project Fits

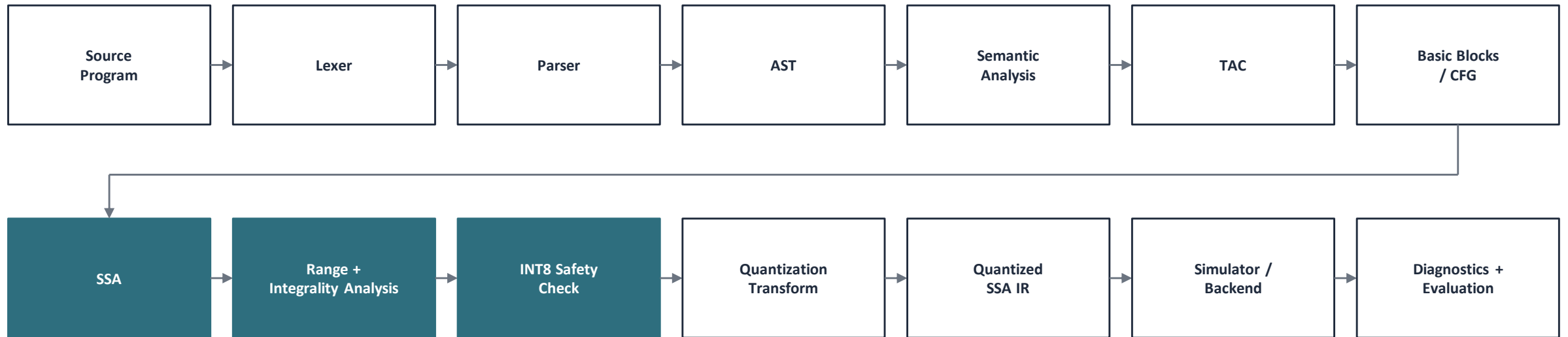
Existing industrial and research systems for quantization typically operate at a substantially larger model, compiler, or framework scale (e.g. whole deep-learning compilers or training frameworks).

This project instead focuses on a small, explainable compiler pipeline in which every quantization decision is directly justified by SSA structure, static range analysis, and integrality checking — making the underlying optimization principle easy to inspect and verify.



Explainable compiler IR + static safety analysis + safe precision reduction

Compiler Pipeline Architecture



Shaded stages form the core static-analysis contribution of this project.

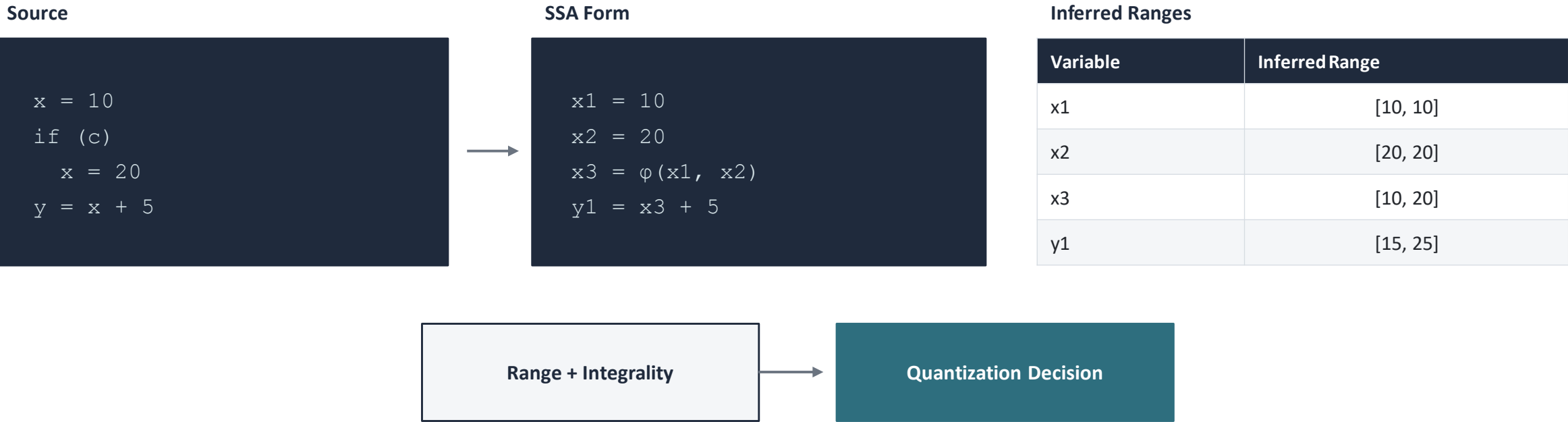
Mapping to BCSE307L Concepts

The pipeline directly exercises core compiler-design concepts from the course syllabus:

BCSE307L Topic	Project Component
Intermediate Code Generation	Three Address Code (TAC)
Control Flow / Backpatching	Control Flow Graph (CFG) construction
Data-Flow Analysis	Static range & integrality analysis
Basic Blocks	CFG partitioning for analysis
Machine-Independent Optimization	Quantization transformation pass
Static Single Assignment (SSA)	SSA intermediate representation
Code Generation	Backend / lightweight simulator

Only concepts that genuinely contribute to the design are shown.

SSA Construction & Range Propagation



Range analysis propagates interval bounds through SSA operations; ϕ -nodes conservatively merge the ranges of incoming branches (union of bounds), so a value's range after a branch always covers every path that reaches it.

Safe vs. Unsafe Quantization

CASE A — SAFE

- $x \in [10, 20]$, integral = true
- $10 \geq -128$ ✓
- $20 \leq 127$ ✓

→ SAFE: FP32 → INT8

CASE B — UNSAFE (Overflow)

- $x \in [120, 200]$, integral = true
- $200 > 127$ ✗ (out of INT8 range)

→ REJECT: retain FP32

Edge Case — Integrity Matters

$x = 1.5$ → range fits numerically, BUT integral = false → ✗ REJECT

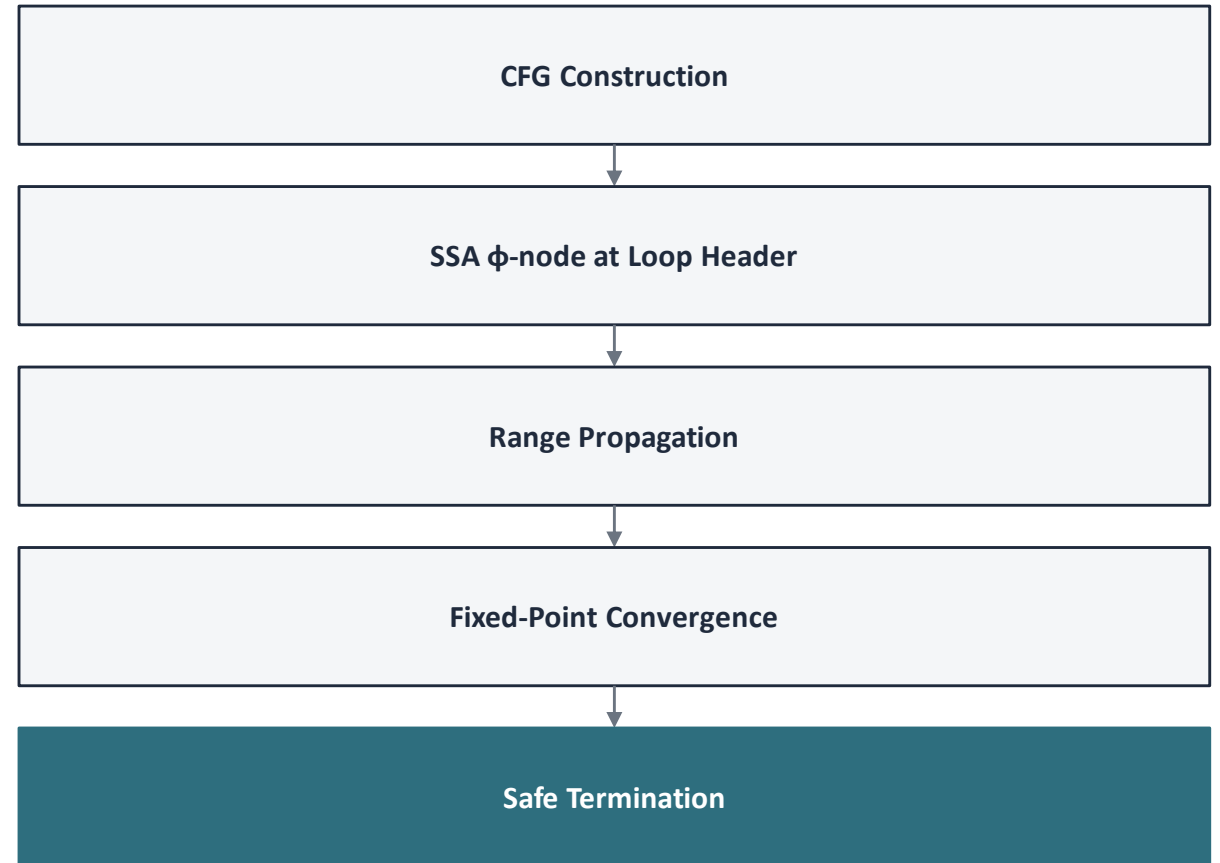
Handling Loops Safely

Example

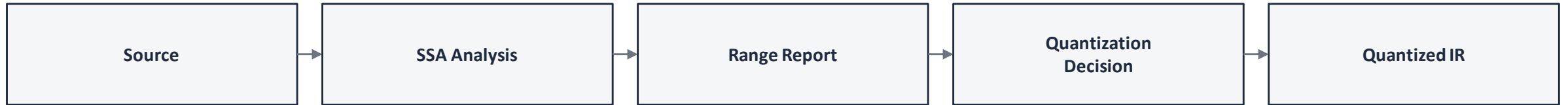
```
i = 0
while (i < 100)
    i = i + 1
```

Loop ranges must be computed via fixed-point iteration; naive propagation may not terminate.

A bounded / widening strategy guarantees the analysis converges to a conservative range in finite time.



Sample Compiler Diagnostics



```
Variable: z  
Source: line 18, column 9  
Range: [30, 60]  
Integral: YES  
INT8 Safety: PASS  
Transformation: FP32 → INT8
```

```
Variable: x  
Source: line 14, column 7  
Range: [120, 200]  
Integral: YES  
INT8 Safety: FAIL  
Reason: possible overflow
```

Project Outcome

The compiler produces three concrete outcomes — each one backed by a compile-time proof rather than a runtime check.

1

Static Safety Check

Verifies at compile time that a value stays integral and remains within $[-128, 127]$ across all execution paths.

2

Selective Quantization

Rewrites only certified-safe operations into INT8, leaving every uncertain operation untouched in FP32.

3

Traceable Output

Emits line-by-line diagnostics stating whether each variable was converted or rejected, and the specific reason.

4×

Reduction in memory per quantized value — 4 bytes (FP32) $\rightarrow$ 1 byte (INT8), with faster integer arithmetic on edge hardware and no risk of a runtime overflow.

Memory reduction follows directly from the width change; runtime and energy gains remain to be measured.

References

- [1] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, “Quantization and training of neural networks for efficient integer-arithmetic-only inference,” in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), 2018, pp. 2704–2713.
- [2] R. Krishnamoorthi, “Quantizing deep convolutional networks for efficient inference: A whitepaper,” arXiv:1806.08342, 2018.
- [3] F. Bourdoncle, “Efficient chaotic iteration strategies with widenings,” in Formal Methods in Programming and Their Applications, LNCS vol. 735, Springer, 1993, pp. 128–141.
- [4] Z. Zhou, X. Chen, E. Li, L. Zeng, K. Luo, and J. Zhang, “Edge intelligence: Paving the last mile of artificial intelligence with edge computing,” Proc. IEEE, vol. 107, no. 8, pp. 1738–1762, 2019.
- [5] T. Chen et al., “TVM: An automated end-to-end optimizing compiler for deep learning,” in Proc. 13th USENIX Symp. on Operating Systems Design and Implementation (OSDI), 2018, pp. 578–594.
- [6] R. Cytron, J. Ferrante, B. K. Rosen, M. N. Wegman, and F. K. Zadeck, “Efficiently computing static single assignment form and the control dependence graph,” ACM Trans. Program. Lang. Syst., vol. 13, no. 4, pp. 451–490, 1991.
- [7] P. Cousot and R. Cousot, “Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints,” in Proc. 4th ACM SIGACT-SIGPLAN Symp. on Principles of Programming Languages (POPL), 1977, pp. 238–252.
- [8] C. Lattner and V. Adve, “LLVM: A compilation framework for lifelong program analysis and transformation,” in Proc. Int. Symp. on Code Generation and Optimization (CGO), 2004, pp. 75–86.
- [9] B. Blanchet, P. Cousot, R. Cousot, J. Feret, L. Mauborgne, A. Miné, D. Monniaux, and X. Rival, “A static analyzer for large safety-critical software,” in Proc. ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI), 2003, pp. 196–207.

Consistent with the reference list in the accompanying Project Review Document.